ОСЬ

Отбор в команду разработки

Тестовое задание

Backend Trainee · Python / FastAPI

Уровень: Trainee / Junior Backend Developer
Стек: Python 3.11+, FastAPI, Pydantic v2, SQLAlchemy 2.0
Формат сдачи: репозиторий на GitHub + README

Это задание проверяет не знание алгоритмов, а подход к работе: как ты раскладываешь код по слоям, проектируешь API, обрабатываешь ошибки и пишешь сервис, который будет удобно поддерживать коллегам.

Нужно собрать REST API для онлайн-бронирования столика в ресторане. Это небольшая, но жизненная продуктовая задача: в ней есть валидация, бизнес-правила и конфликты данных. Делай так, как сделал бы в настоящем проекте.

> ** Не нужно успеть всё. Сначала сделай обязательную часть, бонусы — по желанию. Лучше меньше, но аккуратно.*

1. Задача

Реализуй REST API, которое позволяет создать бронь, получить список броней, получить бронь по id и отменить её.

Эндпоинты

Метод	Путь	Описание
POST	/bookings	Создать бронь. Валидирует данные, возвращает 201 и объект брони.
GET	/bookings	Список броней, отсортированный по дате и времени. Фильтр по дате: ?date=2026-12-15. Ответ: {"items": [...], "total": N}.
GET	/bookings/{id}	Получить бронь по id. 404, если не найдена.
DELETE	/bookings/{id}	Отменить бронь: статус меняется на cancelled, запись физически не удаляется. Возвращает 200 и бронь.

Данные брони

Поле	Тип	Правило
name	str	Имя гостя. 2–50 символов после обрезки пробелов по краям; кириллица, латиница, пробел и дефис.
phone	str	Российский номер: +7XXXXXXXXXX или 8XXXXXXXXXX (10 цифр после кода). При вводе допустимы пробелы, скобки и дефисы. Хранится и возвращается в виде +7XXXXXXXXXX.
booking_date	date	От сегодняшнего дня до сегодня + 90 дней включительно. «Сегодня» считается по Москве (Europe/Moscow).
booking_time	time	Только целые часы 12:00, 13:00 … 22:00, формат HH:MM. Значения вроде 12:30 или 12:00:30 → 422. Для сегодняшней даты нельзя выбрать уже прошедший слот.
guests	int	Целое число от 1 до 12.

Бизнес-правила

- В ресторане 10 столов, размер стола не учитываем. На один слот (дата + время) — не больше 10 активных броней; следующая бронь получает 409.
- Отменённая бронь слот не занимает: после отмены на это время снова можно забронировать.
- Повторная отмена уже отменённой брони возвращает 409.
- Ответ API всегда содержит id, status (active | cancelled) и created_at.

2. Технические требования

Обязательный стек

- Python 3.11+
- FastAPI: роутинг, зависимости, обработка ошибок через HTTPException / exception_handler
- Pydantic v2: схемы запросов и ответов, валидация полей
- Хранение: SQLite через SQLAlchemy 2.0 (async) или in-memory — на твой выбор, на оценку не влияет
- Uvicorn в качестве ASGI-сервера

Будет плюсом

- Alembic для миграций (если выбран SQLAlchemy)
- pytest + httpx.AsyncClient для тестов эндпоинтов
- Dockerfile / docker-compose для локального запуска
- Poetry или requirements.txt с зафиксированными версиями
- Разделение на слои: роуты / сервисы / схемы / модели

Примерная структура проекта

```
app/
  main.py
  api/
    bookings.py          # роуты
  schemas/
    booking.py           # Pydantic-схемы
  services/
    booking_service.py   # бизнес-логика, проверка слотов
  models/
    booking.py           # ORM-модель или dataclass
  core/
    config.py
tests/
  test_bookings.py
```

Пример схемы — schemas/booking.py

```
from datetime import date, datetime, time
from enum import StrEnum
from pydantic import BaseModel, ConfigDict, Field, field_validator

class BookingStatus(StrEnum):
    ACTIVE = 'active'
    CANCELLED = 'cancelled'

class BookingBase(BaseModel):
    name: str = Field(min_length=2, max_length=50, examples=['Анна Петрова'])
    phone: str = Field(examples=['+79991234567'])
    booking_date: date
    booking_time: time = Field(examples=['19:00'])
    guests: int = Field(ge=1, le=12)

class BookingCreate(BookingBase):
    # Валидаторы входных данных: имя, телефон,
    # дата в пределах 90 дней, допустимый слот.
    ...

class BookingOut(BookingBase):
    # Наследуется от BookingBase, а не от BookingCreate:
    # иначе прошлые брони не пройдут проверку даты при чтении.
    model_config = ConfigDict(from_attributes=True)

    id: int
    status: BookingStatus
    created_at: datetime

class BookingList(BaseModel):
    items: list[BookingOut]
    total: int
```

Пример валидатора телефона

```
import re

PHONE_RE = re.compile(r'(\+7|8)\d{10}')

def normalize_phone(value: str) -> str:
    cleaned = re.sub(r'[\s()\-]', '', value)
    if not PHONE_RE.fullmatch(cleaned):
        raise ValueError('Введите номер в формате +7XXXXXXXXXX или 8XXXXXXXXXX')
    return '+7' + cleaned[-10:]
```

** Правильного способа хранения данных здесь нет. Важно, чтобы API было типизировано, проверяло входные данные и возвращало предсказуемые ошибки.*

3. Валидация и обработка ошибок

Ошибки валидации возвращаются в стандартном формате FastAPI (422 Unprocessable Entity), в том числе для нечислового id. Бизнес-ошибки — в формате {"detail": "..."}.

Ситуация	Код	Поведение
Бронь успешно создана	201	Возвращает созданную бронь с id и status=active.
Невалидные данные (телефон, дата, guests и т. п.)	422	Стандартный ответ валидации FastAPI/Pydantic.
Бронь с указанным id не найдена	404	{"detail": "Booking not found"}
На выбранный слот уже 10 активных броней	409	{"detail": "Slot is fully booked"}
Бронь успешно отменена	200	Возвращает бронь со status=cancelled.
Бронь уже отменена	409	{"detail": "Booking is already cancelled"}

Документация

- Swagger UI доступен на /docs, ReDoc — на /redoc (стандартное поведение FastAPI, дополнительно настраивать ничего не нужно)
- У полей Pydantic-схем понятные имена; по желанию — примеры через Field(..., examples=[...])

4. Как сдать задание

Формат

- Публичный репозиторий на GitHub
- Инструкция по запуску (локально и/или через Docker)
- Ссылка на репозиторий — рекрутеру ОСЬ

README.md — обязательно

- Как запустить проект локально (команды)
- Какие решения ты принял и почему (коротко, 3–5 предложений)
- Что бы ты доделал, будь у тебя больше времени
- Если делал бонусы — какие и как (например, чем решил проблему гонки)

** README входит в оценку: умение объяснить свои решения важно не меньше самого кода.*

Чего делать не нужно

- Подключать аутентификацию и авторизацию
- Разворачивать проект на реальном хостинге
- Использовать PostgreSQL или другую внешнюю БД — SQLite или in-memory достаточно
- Добиваться 100% покрытия тестами: тесты — бонус, но хотя бы несколько тестов на ключевые правила заметно усилят решение

5. Критерии оценки

Итоговая оценка — не сумма галочек. Мы смотрим на общее впечатление: насколько с кодом приятно работать и подумал ли ты о разработчике, который откроет проект после тебя.

Критерий	Вес	На что смотрим
Качество кода	30%	Читаемость, разделение на слои, отсутствие дублирования, осмысленные имена
Типизация и Pydantic-схемы	20%	Type hints везде, уместное использование Pydantic, без лишних Any
Логика и валидация	20%	Все правила работают, крайние случаи обработаны, слоты не пересекаются
Обработка ошибок и статус-коды	15%	Коды соответствуют ситуации, сообщения об ошибках понятны
README	15%	Понятное описание, объяснены решения, указано, что осталось доделать

Бонусные баллы

- Async SQLAlchemy 2.0 + миграции Alembic
- pytest-тесты на сервисный слой и эндпоинты (httpx.AsyncClient)
- Dockerfile / docker-compose с запуском одной командой
- Пагинация списка: limit (1–100, по умолчанию 20) и offset; total — общее число броней с учётом фильтра
- Защита от гонки: две одновременные брони на последнее место в слоте не должны пройти обе (транзакция, блокировка или ограничение в БД)
- Линтер и проверка типов: ruff, mypy